deepseq

deepseq: https://hackage.haskell.org/package/deepseq-1.4.8.0

```
-- Control.DeepSeq
class NFData a where
  rnf :: a -> ()

deeqseq :: NFData a => a -> b -> b
force :: NFData a => a -> a
($!!) :: NFData a => (a -> b) -> a -> b
<$!!> :: (Monad m, NFData a) => (a -> b) -> m a -> m b
```

Example: basic usage
Automatically derives instance

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE DeriveAnyClass #-}

import Control.DeepSeq

data MyData = MyData String
  deriving anyclass NFData

-- e.g.
force (MyData "a")
```

==========

Safe exceptions

safe-exceptions: https://hackage.haskell.org/package/safe-exceptions
Exception best practices: https://www.fpcomplete.com/blog/exceptions-best-practices-haskell/

==========

Time

Hackage: https://hackage.haskell.org/package/time-1.12.2

```
-- Data.Time.Clock
-- UTC
data UTCTime = UTCTime { utctDay :: Day, utctTime ::
DiffTime }
instance ParseTime UTCTime, FormatTime UTCTime
data NominalDiffTIme
                  -- e.g. 1 :: NominalDiffTime
instance ParseTime NominalDiffTime, FormatTime
```

```
NominalDiffTime
  Num NominalDiffTime, Real NominalDiffTime, ...

-- UniversalTime
data UniversalTime = ModJulianDate { getModJulianDate ::
Rational }
instance ParseTime UniversalTime,
FormatTime UniversalTime
data DiffTime
                 -- e.g. 1 :: DiffTIme
instance Num DiffTime, ParseTime DiffTime, FormatTime
DiffTIme

getCurrentTime :: IO UTCTime


-- Data.Time.Clock.POSIX
type POSIXTIME = NominalDiffTime
getPOSIXTime :: IO POSIXTime
posixSecondsToUTCTime :: POSIXTime -> UTCTime
utcTimeToPOSIXSeconds :: UTCTime -> POSIXTime
systemToPOSIXTime :: SystemTime -> POSIXTime


-- Data.Time.Clock.System
systemEpochDay :: Day
getSystemTime :: IO SystemTime
systemToUTCTime :: SystemTime -> UTCTime
utcToSystemTime :: UTCTime -> SystemTime
```

Data.Time.Calendar

```
-- Data.Time.Calendar
newtype Day = ModifiedJulianDay Integer
instance DayPeriod Day, ParseTime Day, FormatTime Day,
ISO8601 Day

-- Year
type Year = Integer
pattern CommonEra :: Integer -> Year
pattern BeforeCommonEra :: Integer -> Year

-- Month
type MonthOfYear = Int
pattern January :: MonthOfYear
pattern February :: MonthOfYear
pattern March :: MonthOfYear
pattern April :: MonthOfYear
pattern May :: MonthOfYear
pattern June :: MonthOfYear
pattern July :: MonthOfYear
pattern August :: MonthOfYear
pattern September :: MonthOfYear
pattern October :: MonthOfYear
```

```
pattern November :: MonthOfYear
pattern December :: MonthOfYear

-- Gregorian calendar
type DayOfMonth = Int
toGregorian :: Day -> (Year, MonthOfYear, DayOfMonth)
fromGregorian :: Year -> MonthOfYear -> DayOfMonth ->
Day
pattern YearMonthDay :: Year -> MonthOfYear ->
DayOfMonth -> Day
fromGregorianValid :: Year -> MonthOfYear -> DayOfMonth
-> Maybe Day
showGregorian :: Day -> String
gregorianMonthLength :: Year -> MonthOfYear ->
DayOfMonth
addGregorianMonthsClip :: Integer -> Day -> Day
addGregorianMonthsRollOver :: Integer -> Day -> Day
addGregorianYearsClip :: Integer -> Day -> Day
addGregorianYearsRollOver :: Integer -> Day -> Day
addGregorianDurationClip :: CalendarDiffDays -> Day ->
Day
addGregorianDurationRollOver :: CalendarDiffDays -> Day
-> Day
diffGregorianDurationClip :: Day -> Day ->
CalendarDiffDays
diffGregorianDurationRollOver :: Day -> Day ->
CalendarDiffDays
isLeapYear :: Year -> Bool

-- DayOfWeek
data DayOfWeek = Monday | Tuesday ...
dayOfWeek :: Day -> DayOfWeek
dayOfWeekDiff :: DayOfWeek -> DayOfWeek -> Int
firstDayOfWeekOnAfter :: DayOfWeek -> Day -> Day
weekAllDays :: DayOfWeek -> Day -> [Day]
weekFirstDay :: DayOfWeek -> Day -> Day
weekLastDay :: DayOfWeek -> Day -> Day



-- Data.Time.OrdinalDate
fromOrdinalDate :: Year -> DayOfYear -> Day
fromOrdinalDateValid :: Year -> DayOfYear -> Maybe Day
toOrdinalDate :: Day -> (Year, DayOfYear)
fromMondayStartWeek :: Year -> WeekOfYear -> Int -> Day
fromSundayStartWeek :: Year -> WeekOfYear -> Int -> Day
isLeapYear :: Day -> Bool
...
```

Data.Time.LocalTime

```
-- Data.Time.LocalTime
data TimeZone = { timeZoneMinutes::Int,
timeZoneName::String, timeZoneSummerOnly::Bool }
instance ParseTime TimeZone, FormatTime TimeZone,
```

```
ISO8601 TimeZone

data TimeOfDay = TimeOfDay { todHour:: Int, todMinute::
Int, todSec::Pico }
instance ParseTime TimeOfDay, FormatTime TimeOfDay,
ISO8601 TimeOfDay

data LocalTime = LocalTime { localDay::Day,
localTimeOfDay::TimeOfDay }
instance ParseTime LocalTime, FormatTime LocalTime,
ISO8601 LocalTime

data ZonedTime = ZonedTime
{ zonedTimeToLocalTime::LocalTime,
zonedTimeZone::TimeZone }
instance ParseTime ZonedTime, FormatTime ZonedTime,
ISO8601 ZonedTime

utc :: TimeZone
getTimeZone :: UTCTime -> IO TimeZone
getCurrentTimeZone :: IO TimeZone
utcToLocalTime :: TimeZone -> UTCTime -> LocalTime
localTimeToUTC :: TimeZone -> LocalTime -> UTCTime
getZonedTime :: IO ZonedTime
utcToZonedTime :: TimeZone -> UTCTime -> ZonedTime
utcToLocalZonedTime :: UTCTime -> IO ZonedTime
zonedTimeToUTC :: ZonedTime -> UTCTime
```

Data.Time.Format

```
data TimeLocale = TimeLocale { wDays ::
[(String,String)] ... }
defaultTimeLocale :: TimeLocale

class FormatTime t where ...
formatTime :: FormatTime t => TimeLocale -> String -> t
-> String

class ParseTime t where ...
parseTimeM :: (MonadFail m, ParseTime t) => Bool ->
TimeLocale -> String -> String -> m t
parseSTime :: ParseTime t => Bool -> TimeLocale ->
String -> String -> ReadS t
parsePTime :: ParseTime t => Bool -> TimeLocale ->
String -> String -> ReadP t
parseTimeOrError :: ParseTime t => Bool -> TimeLocale ->
String -> String -> t
```

Types of time

UniversalTime	Universal Time (UT) standard
DiffTime	Length of time by UT

UTC	UTC time
NominalDiffTime	Length of time by UTC
Day	Day from 1858-11-17 e.g. 0 = 1858-11-17
MonthOfYear = Int	1 ~ 12
Year = Int	
DayOfWeek	Monday, Tuesday, ...
POSIXTime = NominalDiffTime	Unix time
SystemTime	Time on machine
TimeOfDay	Hour + minute + second
LocalTime	Day + TimeOfDay
TimeZone	Offset from UTC e.g. JST = UTC+9:00
ZonedTime	LocalTime + TimeZone

Formatting

Formatting: https://hackage.haskell.org/package/time-1.12.2/docs/Data-Time-Format.html#g:1

%F	%Y-%m-%d
%D	%m/%d/%y
%Y	e.g. 2022
%y	e.g. 22
%B	January-December
%b	Jan-Dec
%m	01-12
%d	01-31
%e	1-31
%j	001-366
%R	%H:%M
%T	%H:%M:%S
%H	00-23
%I	01-12
%P	am/pm
%p	AM/PM
%M	00-59
%S	00-60

```
---- Times
--
time <- getCurrentTime            -- UTC
 e.g. 2023-05-07 03:00:00 UTC
time <- getZonedTime              -- Current location
 e.g. 2023-05-07 12:00:00 JST


-- Construction
toEnum 1 :: Day             -- 1858-11-18 i.e. 1 day
from 1858-11-17
1 :: DiffTime               -- 1s
1 :: NominalDiffTime        -- 1s
UTCTime (toEnum 1) 1        -- 1858-11-18 00:00:01 i.e.
1 day + 1s
```

```
---- Day
--
type Year = Integer
type DayOfYear = Int
fromOrdinalDate :: Year -> DayOfYear -> Day
fromOrdinalDateValid :: Year -> DayOfYear -> Maybe Day
toOrdinalDate :: Day -> (Year, DayOfYear)
isLeapYear :: Day -> Bool
fromMondayStartWeek :: Year -> WeekOfYear -> Int -> Day
fromSundayStartWeek :: Year -> WeekOfYear -> Int -> Day
fromOrdinalDate 2020 1                -- Day: 2020-01-01
fromOrdinalDate 2020 32               -- Day: 2020-02-01
fromMondayStartOfWeek 2020 1 1        -- Day:
2020-01-06 i.e. 1st Monday of 2020
fromMondayStartOfWeek 2020 2 1        -- Day:
2020-01-13 i.e. 2nd Monday of 2020
fromMondayStartOfWeek 2020 1 2        -- Day:
2020-01-07 i.e. 1st Tuesday of 2020


---- Parse/Format
--
import Data.Time.Format
import Data.Time.Calendar
parseTimeM :: (MonadFail m, ParseTime t) =>
  Bool                      -- Ignore leading/trailing
spaces
  -> TimeLocale
  -> String                 -- format string
  -> String                 -- string to parse
  -> m t
formatTime :: FormatTime t => TimeLocale -> String ->
t -> String
d <- parseTimeM False
defaultTimeLocale "%F" "2020-01-01" :: IO Day        --
Day: 2020-01-01
formatTime defaultTimeLocale "%F" d
                     -- "2020-01-01"


---- Parse ISO8601
--
import Data.Time.Format.ISO8601
t :: UTCTime <-
formatParseM  (iso8601Format)
"2023-01-01T01:01:01.111Z"      -- UTC yyyy-mm-
ddThh:mm:ss[.sss]Z
t :: Day <-
formatParseM  (iso8601Format)  "2023-01-01"
             -- Day yyyy-mm-dd
```

Usage: Create timestamp
Use **getZonedTime** to get local time

```
import Data.Time

main :: IO ()
main = do
 timestamp <- formatTime defaultTimeLocale "%T" <$>
getZonedTime        -- 01:02:03
 timestamp <- formatTime defaultTimeLocale "%T" <$>
getCurrentTime      -- OK, but UTC time,
 timestamp <- formatTime defaultTimeLocale "%F %T" <$>
getZonedTime     -- 2023-01-01 01:02:03
```

==========
======

Extra

Hackage: https://hackage.haskell.org/package/extra-1.7.12

```
Data.Either.Extra
fromLeft :: a -> Either a b -> a
fromRight :: b -> Either a b -> b
eitherToMaybe :: Either a b -> Maybe b
maybeToEither :: a -> Maybe b -> Either a b
... etc


Control.Exception.Extra
Control.Monad.Extra
... etc
```